

zpluscrm & System Architecture

Supabase Database Operations Full Flow Guide (Insert, Update, Delete)

Project ID: dwtsntjkyxslqluouhbr

SDK & REST API: PostgREST / Supabase JS v2

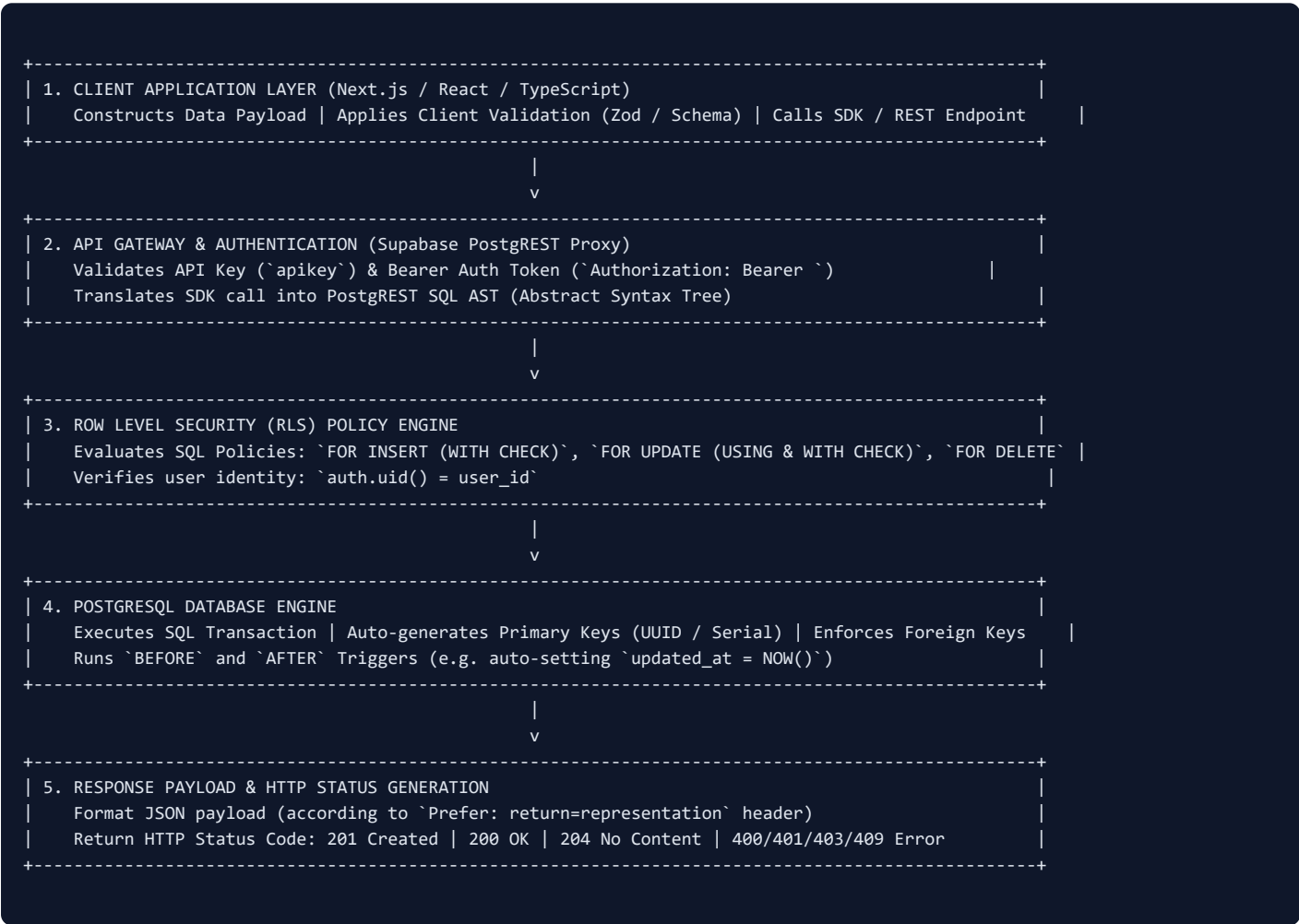
Updated: August 2026

📌 Document Purpose

This reference manual details the **complete end-to-end execution flow** for database operations (`INSERT` , `UPDATE` , `DELETE`) in Supabase. It covers client payload preparation, SDK & REST API calls, Row Level Security (RLS) enforcement, database triggers, status response codes, and production code patterns.

1. System Architecture & End-to-End Execution Pipeline

When an application executes a database mutation in Supabase (whether via standard JavaScript/TypeScript SDK or raw HTTP cURL calls), the request traverses six distinct layers:



2. INSERT Operation — Complete Full Flow (Create Data)

The `INSERT` operation creates one or multiple new rows in a target database table.

INSERT Lifecycle Flow Breakdown

1 Payload Preparation & Schema Validation

The application constructs a JavaScript object containing field values. Optional fields get default database values if omitted.

2 SDK Call / HTTP POST Dispatch

Calls `supabase.from('table').insert([data])` or sends HTTP `POST /rest/v1/table` with `Content-Type: application/json`.

3 Header Inspection & Authentication

PostgREST inspects `apikey` and `Authorization: Bearer <JWT>` to resolve the active user session ID (`auth.uid()`).

4 Row Level Security (RLS) Policy Check

Evaluates `INSERT WITH CHECK (...)` policies. If policy fails, execution aborts with `403 Forbidden / 42501 RLS Violation`.

5 PostgreSQL Storage & Primary Key Generation

PostgreSQL generates primary key UUID/ID, evaluates column constraints (UNIQUE, NOT NULL), populates `created_at = NOW()`, and commits row.

6 HTTP Response & Payload Return

Returns HTTP `201 Created`. If header `Prefer: return=representation` is set, returns the newly inserted record JSON.

A. Supabase JavaScript SDK Insert & Upsert Code

```
import { supabase } from "../supabase";

// 1. Standard Insert Operation
export async function createNewClient(clientData) {
  const { data, error } = await supabase
    .from("clients")
    .insert([
      {
        id: `c_${Date.now()}`,
        name: clientData.name,
        type: clientData.type, // e.g. "PRIVATE_LIMITED"
        email: clientData.email,
        mobile: clientData.mobile,
        user_id: clientData.user_id
      }
    ])
    .select(); // Returns created representation

  if (error) {
    console.error("Insert failed:", error.message);
    throw error;
  }
  return data[0];
}

// 2. Upsert Operation (Insert or Update if Conflict exists)
export async function upsertClient(clientRecord) {
  const { data, error } = await supabase
    .from("clients")
    .upsert(clientRecord, { onConflict: "id" })
    .select();
  return { data, error };
}
```

B. Supabase REST API / cURL Insert Command

```
curl -X POST "https://dwtsntjkyxslqluouhbr.supabase.co/rest/v1/clients" \  
-H "apikey: $SUPABASE_ANON_KEY" \  
-H "Authorization: Bearer $SUPABASE_ANON_KEY" \  
-H "Content-Type: application/json" \  
-H "Prefer: return=representation" \  
-d '{  
  "id": "c_172400100",  
  "name": "Apex Global Tech",  
  "type": "PRIVATE_LIMITED",  
  "mobile": "9876543210",  
  "email": "info@apexglobal.com",  
  "city": "Mumbai",  
  "user_id": "usr_harsha_g_c"  
'
```

C. PostgreSQL RLS Insert Security Policy

```
-- Policy: Allow users to insert records only if user_id matches their Auth ID  
CREATE POLICY "Enable insert for authenticated users"  
ON clients FOR INSERT  
WITH CHECK (auth.uid() = user_id OR user_id = 'usr_harsha_g_c');
```

3. UPDATE Operation — Complete Full Flow (Modify Data)

The `UPDATE` operation modifies existing row(s) matching filter criteria.

UPDATE Lifecycle Flow Breakdown

1 Target Row Selection & Patch Payload Assembly

Client identifies record by primary key or filter condition (e.g. `id = c_101`) and passes modified field key-value pairs.

2 SDK Dispatch / HTTP PATCH Request

Calls `supabase.from('table').update({ field: val }).eq('id', targetId)` or HTTP `PATCH /rest/v1/table?id=req.targetId`.

3 RLS Policy Dual Check (USING & WITH CHECK)

Evaluates `USING (...)` (can user see existing row?) and `WITH CHECK (...)` (is user allowed to write new values?).

4 Trigger Execution ('BEFORE UPDATE')

PostgreSQL executes active triggers (e.g. `BEFORE UPDATE ON table FOR EACH ROW EXECUTE FUNCTION update_timestamp()`).

5 PostgreSQL Disk Write & Index Maintenance

Applies partial row updates, updates modified column indexes, and commits transaction.

6 HTTP Response Delivery

Returns `200 OK` with updated JSON representation (or `204 No Content` if return header omitted).

A. Supabase JavaScript SDK Update Code

```
import { supabase } from "../supabase";

export async function updateLeadStatus(leadId, newStatus, notes) {
  const { data, error } = await supabase
    .from("leads")
    .update({
      status: newStatus, // "CONTACTED", "QUALIFIED", "WON", "LOST"
      notes: notes,
      updated_at: new Date().toISOString()
    })
    .eq("id", leadId) // Strict filter predicate
    .select();

  if (error) {
    console.error("Update failed:", error.message);
    return null;
  }
  return data[0];
}
```

B. Supabase REST API / cURL Update Command

```
curl -X PATCH "https://dwtstntjkysxlqluouhbr.supabase.co/rest/v1/leads?id=req.lead_301" \
-H "apikey: $SUPABASE_ANON_KEY" \
-H "Authorization: Bearer $SUPABASE_ANON_KEY" \
-H "Content-Type: application/json" \
-H "Prefer: return=representation" \
-d '{
  "status": "QUALIFIED",
  "value": 45000,
  "notes": "Client requested proposal for annual tax filing."
}'
```

C. PostgreSQL Automatic `updated\_at` Trigger SQL

```
-- Trigger function to keep updated_at current automatically
CREATE OR REPLACE FUNCTION set_updated_at_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_leads_timestamp
BEFORE UPDATE ON leads
FOR EACH ROW EXECUTE FUNCTION set_updated_at_timestamp();
```

4. DELETE Operation — Complete Full Flow (Remove Data)

The `DELETE` operation removes row(s) from a database table. In enterprise applications, deletion is categorized into **Hard Deletes** (permanent SQL row removal) and **Soft Deletes** (marking `is_deleted = true`).

DELETE Lifecycle Flow Breakdown

1 Deletion Criteria Predicate Definition

Client sets strict target primary key filter (e.g., `id = inv_501`) to prevent unintended mass deletion.

2 SDK Dispatch / HTTP DELETE Dispatch

Calls `supabase.from('table').delete().eq('id', id)` or HTTP `DELETE /rest/v1/table?id=req.id`.

3 RLS Policy USING Verification

PostgreSQL evaluates `DELETE USING (...)` policy. If caller is not row owner, delete call affects 0 rows or throws permission error.

4 Foreign Key Constraint Enforcement

Database checks child table references. If child records exist without `ON DELETE CASCADE`, PostgreSQL aborts with `23503 FK Violation`.

5 PostgreSQL Physical Deletion / Cleanup

Removes tuple from database pages, updates table indexes, and frees disk space via vacuuming.

6 Response Delivery & Status Code

Returns `200 OK` (with deleted record JSON if requested) or `204 No Content`.

A. Hard Delete vs Soft Delete SDK Code Patterns

```
import { supabase } from "../supabase";

// Pattern 1: Hard Delete (Permanent Database Removal)
export async function hardDeleteBankingEntry(entryId) {
  const { data, error } = await supabase
    .from("banking_entries")
    .delete()
    .eq("id", entryId)
    .select();

  if (error) throw error;
  return data;
}

// Pattern 2: Soft Delete (Enterprise Recommended Archival)
export async function softDeleteClient(clientId) {
  const { data, error } = await supabase
    .from("clients")
    .update({
      is_deleted: true,
      deleted_at: new Date().toISOString()
    })
    .eq("id", clientId)
    .select();

  return { data, error };
}
```

B. Supabase REST API / cURL Delete Command

```
curl -X DELETE "https://dwtstntjkysxlqluouhbr.supabase.co/rest/v1/banking_entries?id=eq.bnk_901" \  
-H "apikey: $SUPABASE_ANON_KEY" \  
-H "Authorization: Bearer $SUPABASE_ANON_KEY" \  
-H "Prefer: return=representation"
```

C. PostgreSQL RLS Delete Policy

```
-- Restrict deletion strictly to record owner  
CREATE POLICY "Enable delete for owner"  
ON banking_entries FOR DELETE  
USING (auth.uid() = user_id);
```

5. HTTP Status Codes & Error Handling Matrix

The matrix below summarizes standard PostgREST / Supabase response status codes during database operations:

Code	Status Title	Operation	Meaning & Cause	Action Required
201	Created	INSERT	Record created successfully. Payload returned if <code>return=representation</code> set.	Proceed with application flow.
200	OK	UPDATE / DELETE	Operation completed successfully and returned affected record(s).	Update UI state.
204	No Content	UPDATE / DELETE	Operation succeeded without returning body payload.	Confirm success in UI.
400	Bad Request	ALL	Schema error (invalid data type, missing non-null field, malformed UUID).	Validate input schema before dispatching.
401	Unauthorized	ALL	Missing or invalid <code>apikey</code> or JWT token in request headers.	Check auth credentials & login session.
403	Forbidden	ALL	Row Level Security (RLS) policy check failed for active <code>user_id</code> .	Verify user permissions & RLS SQL policy.
409	Conflict	INSERT / UPDATE	Duplicate primary key or UNIQUE constraint violation (e.g. existing email).	Use <code>.upsert()</code> or prompt user for unique data.
500	Internal Server Error	ALL	Database server issue, connection failure, or unhandled trigger crash.	Inspect Supabase logs & trigger functions.

Standard TypeScript Production Wrapper Pattern

```
import { supabase } from "../supabase";

export async function executeSupabaseMutation(operationFn) {
  try {
    const { data, error } = await operationFn();

    if (error) {
      // Handle Supabase PostgREST Error Object
      console.warn(`Supabase Mutation Error [Code: ${error.code}]:`, error.message);

      switch (error.code) {
        case "23505": // Unique constraint violation
          throw new Error("A record with this identifier already exists.");
        case "42501": // RLS Policy failure
          throw new Error("You do not have permission to modify this item.");
        default:
          throw new Error(error.message || "Database operation failed.");
      }
    }

    return { success: true, data };
  } catch (err) {
    return { success: false, error: err.message };
  }
}
```


6. Real-World zplusrm Database Operations Reference

A. Inserting a New Service Package (`services`)

```
// Insert new GST Compliance Service package
const { data, error } = await supabase
  .from("services")
  .insert([
    {
      id: "srv_gst_01",
      name: "GST Monthly Filing & Reconciliation",
      category: "TAXATION",
      default_fee: 3500,
      user_id: "usr_harsha_g_c"
    }
  ])
  .select();
```

B. Updating Invoice Payment Status (`invoices`)

```
// Mark Invoice as Paid with Payment Reference
const { data, error } = await supabase
  .from("invoices")
  .update({
    status: "PAID",
    paid_amount: 15400,
    payment_mode: "UPI",
    paid_date: new Date().toISOString().split('T')[0]
  })
  .eq("id", "inv_2026_089")
  .select();
```

C. Deleting a Banking Ledger Entry (`banking\_entries`)

```
// Remove erroneous ledger transaction
const { data, error } = await supabase
  .from("banking_entries")
  .delete()
  .eq("id", "bnk_trn_991")
  .eq("user_id", "usr_harsha_g_c");
```

⚠ Summary Checklist for Supabase Database Operations

- Always send mandatory headers: `apikey` and `Authorization: Bearer <token>`.
- Use `Prefer: return=representation` if you want the mutated row returned in response payload.
- Enforce strict primary key checks (`.eq('id', value)`) on `UPDATE` and `DELETE` calls.
- Enable Row Level Security (RLS) on all production tables to prevent unauthorized data access.
- Prefer **Soft Delete** over Hard Delete for critical compliance or financial entities.